UBUNTU 26.04 · CE FIELD GUIDE · MAY 2026

Linux Command Reference

safwan@Safwan-Lenovo-IdeaPad:~$ — everything you need, nothing you don't

01 Essential Commands

Navigation	description
`pwd`	where am I right now?
`ls`	what's in this folder?
`ls -la`	same + hidden files + permissions + sizes
`cd ~/Downloads`	go to Downloads folder
`cd ..`	go UP one folder in the tree structure
`cd -`	go BACK to previous location (like browser back button)
`cd ~`	go home from anywhere

Creating Things	description
`mkdir foldername`	create a folder
`mkdir -p a/b/c`	create nested folders in one go
`touch file.txt`	create an empty file

Opening & Reading Files	description
`xdg-open file.pdf`	open any file in its default app — like double-clicking
`xdg-open .`	open current folder in Files app
`cat file.txt`	print entire file contents to terminal
`less file.txt`	scroll through file (q to quit)
`nano file.txt`	edit a text file inside terminal

Moving, Copying, Renaming	description
`mv file.txt ~/Documents/`	move file to Documents — same name
`mv file.txt newname.txt`	rename file — same folder
`mv file.txt ~/Documents/new.txt`	move AND rename simultaneously
`mv folder/ ~/Documents/`	move entire folder — no -r needed

`cp file.txt ~/Documents/`	copy file to Documents — same name
`cp file.txt newname.txt`	copy with a different name, same folder
`cp file.txt ~/Documents/new.txt`	copy to different folder with new name
`cp -r folder/ ~/Documents/`	copy entire folder — -r required
`cp -r folder/ ~/Documents/newname/`	copy folder with a new name

■ rm is permanent — no recycle bin. Always double-check the path before running. One wrong character and you've deleted the wrong thing with no undo.

Deleting	description
`rm file.txt`	delete a file — permanent, no recycle bin
`rm -r foldername/`	delete folder and all its contents recursively
`rm -rf foldername/`	same but force — no confirmation prompts
`rmdir foldername/`	delete EMPTY folder only — refuses if contents exist

Software Management — apt	description
`sudo apt update`	refresh package list from repositories
`sudo apt upgrade -y`	install all available updates
`sudo apt install name -y`	install something
`sudo apt remove name -y`	uninstall something
`sudo apt purge name -y`	uninstall and remove all config files
`sudo apt autoremove -y`	clean up unused dependencies

System Info	description
`df -h`	disk space usage (-h = human readable)
`free -h`	RAM usage
`top`	live process monitor like Task Manager (q to quit)
`htop`	prettier top — sudo apt install htop first
`lsblk -f`	list all drives and partitions with filesystem info

Getting Help	description
`man commandname`	full manual for any command (q to quit)
`commandname --help`	quick summary of all options

02 Terminal Shortcuts

`Tab`	Autocomplete filenames and commands. Use obsessively — prevents typos and confirms the file exists.
`Ctrl+R`	Search command history. Start typing part of a previous command and it finds it instantly.
`Up / Down`	Cycle through previous commands one by one.
`Ctrl+C`	Cancel a running command immediately. Your escape hatch when something hangs.
`Ctrl+L`	Clear the terminal screen. Same as typing clear.
`Ctrl+Shift+C`	Copy in terminal — Ctrl+C would cancel the command instead.
`Ctrl+Shift+V`	Paste in terminal.
`!!`	Repeat last command. Type sudo !! to rerun with sudo if you forgot it.
`hash -r`	Clear shell command cache. Fixes command not found after installing something new.

03 Paths — Absolute vs Relative

ABSOLUTE PATH — works from anywhere	RELATIVE PATH — from current location
Starts from / or ~ Like a full postal address ~/Documents/lab3.mlx /mnt/Saftech/Lab 3/ /home/safwan/Downloads Use when in doubt. Always unambiguous.	Starts from where you are now Like turn left at the corner ./file.txt (here) ../file.txt (one up) subfolder/file.txt Shorter but depends on location.

Special Path Symbols

`~` → your home directory = /home/safwan (not /home — that contains ALL users)

`/` → root of entire file system — everything hangs off this

`.` → current directory (./firefox means run firefox from here)

`..` → parent directory — one level up

`*` → wildcard — matches any filename (rm *.txt deletes all .txt files)

You can mix absolute and relative freely

rm, mv, cp all accept either or both in the same command:

```
mv ./localfile.txt ~/Documents/ — relative source, absolute destination
cp /mnt/Saftech/lab.mlx ./copy.mlx — absolute source, relative destination
rm ../somefile.txt — relative path going up one level
```

04 cd .. vs cd — — What's the Difference?

cd .. — structure-based	cd - — history-based
Goes UP one level in folder tree. About WHERE YOU ARE in structure. /mnt/Saftech/Lab 3/Data → cd .. → /mnt/Saftech/Lab 3/ Predictable. Based on folder tree.	Goes BACK to previous location. About WHERE YOU WERE before. You're in ~/Downloads cd /mnt/Saftech/Lab 3/Data cd - → back to ~/Downloads Like browser back button.

05 mv and cp — The Full Picture

Why mv also renames — the real reason

On Linux, a filename is just a label attached to a location in the file system.

Moving and renaming are literally the same operation — you're just changing

what label points to what location.

```
mv old.txt new.txt — same folder, different label = rename
mv old.txt ~/Documents/ — different folder, same label = move
mv old.txt ~/Documents/new.txt — different folder AND label = both at once
```

Why cp needs -r for folders but mv doesn't

cp physically duplicates every file — needs -r to go inside and copy everything

including subfolders of subfolders.

mv just relabels the folder's location — no actual copying. So mv never needs -r.

```
cp -r folder/ ~/Documents/ — -r required
mv folder/ ~/Documents/ — no -r needed — mv is the odd one out
rm -r folder/ — -r required — needs to traverse contents
```

The -r pattern — consistent across commands	description
`cp file`	works fine
`cp -r folder`	needs -r — physically copies all contents
`mv file`	works fine
`mv folder`	works — no -r needed (mv is the odd one out)
`rm file`	works fine
`rm -r folder`	needs -r — must traverse contents to delete
`rmdir folder`	only works if folder is completely EMPTY

06 Wildcards — Selecting Multiple Files at Once

What * actually does

```
* matches any filename or part of a filename. The shell expands it BEFORE
running the command — so rm *.txt becomes rm file1.txt file2.txt lab3.txt
automatically. You never type each filename.

Two important limits:
* does NOT include hidden files (filenames starting with .)
* does NOT recurse into subfolders — only matches current folder level
```

Wildcard Examples	description
`rm *`	delete every file in current folder — folder itself survives
`rm *.txt`	delete only .txt files in current folder
`rm lab*`	delete anything starting with lab
`rm *report*`	delete anything containing the word report
`cp * ~/Documents/`	copy everything to Documents
`mv * ~/Documents/`	move everything to Documents
`ls *.mlx`	list all .mlx files without deleting them

rm * vs rm -r foldername/ — the box analogy

```
rm * — throws everything OUT of the box. The box (folder) remains.

rm -r folder/ — throws the box AND everything inside it away.


rm * expands to a list of files, never the folder container itself.

To also delete the folder: rm * then rmdir foldername/

Or just: rm -r foldername/ from outside the folder — one step.
```

07 Shell Scripting — Why the Terminal is Truly Powerful

Two levels of terminal power

```
LEVEL 1 — Terminal without scripting:

Already faster than GUI. One command replaces many clicks.

mv * ~/Documents/ moves 50 files in one keystroke instead of

selecting all, dragging, and confirming 50 times.


LEVEL 2 — Terminal with scripting:

Orders of magnitude more powerful. Add logic — loops, conditions,

variables — so the terminal makes decisions and repeats operations

automatically. Process thousands of files overnight while you sleep.

Schedule tasks. Chain commands so output of one feeds into another.


Shell scripting = programming but for your OS instead of an app.

Every command you've learned is a building block.
```

Your first shell script — the for loop

```
Rename every .txt file by adding a lab_ prefix:


for f in *.txt; do mv "$f" "lab_$f"; done


Breaking it down:

for f in *.txt — loop through every .txt file, call each one f

do mv "$f" "lab_$f" — rename each one by adding the prefix

done — end the loop


Result:

file1.txt → lab_file1.txt

file2.txt → lab_file2.txt


Conceptually identical to a for loop in Java or Python.

The variable f holds the current filename in each iteration.
```

A real example — batch rename ENGN lab files

```
You have: Lab1.mlx, Lab2.mlx, Lab3.mlx

You want: ENGN0040_Lab1.mlx, ENGN0040_Lab2.mlx, ENGN0040_Lab3.mlx


for f in Lab*.mlx; do mv "$f" "ENGN0040_$f"; done


All renamed instantly. Try doing that in a GUI.
```

Basic scripting building blocks	description
`for f in *.txt; do ... done`	loop through all .txt files
`for f in *; do ... done`	loop through everything in folder
`if [ -f file.txt ]; then ... fi`	do something only if file exists
`command1 && command2`	run command2 only if command1 succeeded
`command1 || command2`	run command2 only if command1 failed
`command1 | command2`	pipe — feed output of command1 into command2

08 find — Search for Files by Properties

What find does vs what grep does

```
find → searches the FILE SYSTEM for files/folders matching criteria.
It never looks inside files. Finds the file itself.

grep → searches for TEXT INSIDE files.
It never cares about where files are. Finds content inside.

Analogy — imagine a library:
find = asking the librarian where is the book called lab3
grep = opening every book and scanning for the word velocity
```

Syntax: `find [WHERE] [FILTER] [ACTION]`

WHERE — can be anywhere on your system	description
`find .`	current folder
`find ~`	your entire home directory
`find /mnt/Saftech`	entire Saftech partition
`find /etc`	all system configuration files
`find /var/log`	all system log files
`find /`	entire system — slow, needs sudo for some dirs

FILTER — by name	description
`-name "*.mlx"`	exact case match — files ending in .mlx
`-iname "*.mlx"`	case insensitive version of -name
`-name "lab*"`	starts with lab
`-name "*report*"`	contains the word report
`-name "*lulu"`	ends with lulu
`-type f`	files only — not folders
`-type d`	directories only — not files

FILTER — by time	description
`-mtime 0`	modified today
`-mtime -7`	modified in the last 7 days
`-mtime +30`	modified more than 30 days ago
`-newermt "2026-04-01"`	modified after April 1st 2026

FILTER — by size	description
`-size +10M`	larger than 10 megabytes
`-size -1M`	smaller than 1 megabyte
`-size +1G`	larger than 1 gigabyte

Combining filters — AND by default	description
`find ~ -name "*.pdf" -mtime -7`	PDFs modified in last 7 days
`find ~ -type f -size +100M`	files larger than 100MB
`find /mnt/Saftech -name "*.mlx" -mtime 0`	.mlx files modified today
`find /mnt/Saftech -name "*lulu" -type f`	files ending in lulu

ACTION — what to do with results	description
`find ~ -name "*.pdf"`	default — print all matching paths
`find ~ -name "*.tmp" -delete`	delete everything found — careful
`find ~ -name "*.mlx" | xargs wc -l`	count lines in each found file
`find ~ -name "*.mlx" | xargs grep "word"`	search inside all found files

09 grep — Search for Text Inside Files

Syntax: grep [FLAGS] "pattern" [WHERE]

What file types grep works on

```
Works well on plain text: .txt .md .py .java .m .c .cpp .html .log .json .tex


Useless or garbage output on binary formats:
.pptx .docx .xlsx — actually ZIP archives containing XML
.blend — Blender binary format
.mlx — MATLAB live scripts are binary/XML hybrid
.pdf — partially works on text-based PDFs but messy


For .pptx and .blend — use the application's own search feature instead.
```

WHERE — can be anywhere	description
`grep "word" file.txt`	search inside one specific file
`grep "word" file1.txt file2.txt`	search inside multiple named files

`grep "word" *.mlx`	search inside all .mlx files here
`grep -r "word" .`	search inside everything in current folder
`grep -r "word" ~`	search inside everything in home directory
`grep -r "word" /mnt/Saftech`	search inside everything on Saftech
`grep -r "word" /var/log`	search inside all system logs
`grep -r "word" /etc`	search inside all system config files

Essential flags	description
`-i`	case insensitive — velocity matches Velocity and VELOCITY
`-r`	recursive — search inside all files in a folder and subfolders
`-n`	show line numbers next to each match
`-l`	show only filenames — not the matching lines themselves
`-L`	show only filenames that DON'T contain the pattern
`-c`	show count of matches per file instead of the lines
`-v`	invert — show lines that do NOT match the pattern
`-w`	whole word only — velocity won't match velocities or prevelocity
`-A 3`	show 3 lines AFTER each match for context
`-B 3`	show 3 lines BEFORE each match for context
`-C 3`	show 3 lines before AND after each match — full context
`--include="*.mlx"`	only search inside files matching this pattern
`--exclude="*.pdf"`	skip files matching this pattern

Practical combinations for CE work	description
`grep -rn "TODO" /mnt/Saftech/`	find all TODOs with line numbers
`grep -ri "velocity" /mnt/Saftech/ --include="*.mlx"`	case insensitive in .mlx only
`grep -rl "integral" /mnt/Saftech/`	which files contain integral — names only
`grep -C 2 "error" /var/log/syslog`	errors with 2 lines of context each side
`grep -v "comment" code.py`	show all lines NOT containing comment
`grep -rc "for" /mnt/Saftech/Lab\ 3/`	count for loops per file in Lab 3
`grep -w "for" code.py`	match for but not format or before

10 Combining find and grep — Full Power

The pattern

```
find [location] [filter] | xargs grep "searchterm"


The | pipes find's results into xargs.

xargs takes that list of files and feeds them one by one into grep.


Why xargs? grep expects filenames as arguments not as piped input.

xargs bridges that gap — converts piped list into arguments.


Without xargs: find ... | grep "word" ← grep searches the LIST OF FILENAMES

With xargs: find ... | xargs grep "word" ← grep searches INSIDE each file
```

Practical combined searches	description
`find /mnt/Saftech -name "*.mlx" | xargs grep "velocity"`	search for velocity inside all .mlx files on Saftech
`find /mnt/Saftech -name "*lulu" | xargs grep "word"`	search inside files ending with lulu
`find /mnt/Saftech -mtime 0 | xargs grep "velocity"`	search inside files modified today only
`find /mnt/Saftech -mtime 0 -name "*.mlx" | xargs grep "velocity"`	.mlx files modified today containing velocity
`find /mnt/Saftech -name "*.mlx" | xargs grep -in "velocity"`	case insensitive with line numbers
`find /mnt/Saftech -size +1M -name "*.pdf" | xargs grep -l "calculus"`	PDFs larger than 1MB containing calculus — show filenames only

When to use grep --include vs find | xargs grep

```
grep -r "word" /mnt/Saftech --include="*.mlx"

→ use when you only need to filter by file type — simpler, one command


find /mnt/Saftech -name "*.mlx" -mtime -7 | xargs grep "word"

→ use when you need time, size, or complex filters — grep can't do those alone
```

Quick decision guide	description
`find a file by name`	find
`find a file by size or date`	find
`search text inside one file`	grep
`search text across many files`	grep -r

search text only in certain file types	grep -r --include
search text in files matching date/size	find | xargs grep

11 How Files Actually Work — The Inode System

What a file actually is

```
Your SSD is a giant sequence of numbered blocks of data. A file's content
is scattered across many non-adjacent blocks anywhere on disk.

What makes a file a file is an entry in a directory table called an INODE —
just a pointer saying this name refers to these specific blocks of data.
The filename is just a label. The inode is the real thing.

SSD blocks (raw data):
[ 4821 ][ 4822 ][ 9103 ][ 9104 ][ 1205 ] ← actual content, scattered

Inode table:
lab3.mlx → blocks 4821, 4822, 9103, 9104, 1205

When you type: mv lab3.mlx newname.mlx
label lab3.mlx REMOVED from inode table
label newname.mlx ADDED pointing to the exact same blocks
The data never moved. Just the label changed.
```

mv WITHIN same drive — instant	mv ACROSS drives — takes time
Just relabels in inode table. Data blocks never touched. INSTANT even for huge files. /mnt/Saftech/a → /mnt/Saftech/b = label change only	Separate file systems — can't relabel. Must physically copy all data blocks then delete original blocks. /mnt/Saftech → /mnt/Mamoni = secretly a cp + rm

12 sudo, Root, and Permissions

The permission model

```
Every file has three permission levels:
Owner — the user who created it, full control
Group — a group of users who share some access
Others — everyone else, usually restricted

/home/safwan — you have full access, everyone else has none
/usr/local/ — only root can write, everyone can read
```

sudo — superuser do

```
Runs ONE command as root then drops back immediately.
Every sudo use is logged with your username — full audit trail.

Linux makes you type sudo explicitly every time — a deliberate speed bump.
On Windows you're often admin by default. On Linux you're always a regular
user unless you explicitly escalate.

sudo command — run one command as root ✓ use this
sudo su — become root permanently ✗ avoid — no speed bump, dangerous
sudo -H ./install — run GUI installer as root (used for MATLAB install)
```

Root — the superuser

```
Root has no permission restrictions. Can read, write, delete anything.

Multiple users can have sudo access — each with their own password.
Any sudo user can do anything including modifying other sudo users' files.
Sudo access is purely a matter of trust.

Security benefit: malware running as your regular user cannot touch system
files without your sudo password. On Windows, admin malware damages everything.
```

13 Linux File System — The Big Picture

Standard directories — what lives where	description
`/`	root — the entire file system hangs off this
`/home`	all users' home directories live here

`~`	shortcut to YOUR home = /home/safwan specifically
`/usr`	installed programs and their files = like C:\Program Files
`/usr/local/bin`	manually installed executables — Firefox symlink lives here
`/etc`	system config files — readable text, unlike Windows registry
`/etc/fstab`	tells Linux how to mount each partition on boot — we edited this
`/tmp`	temporary files — cleared on reboot
`/dev`	hardware devices as files — your SSD = /dev/nvme0n1
`/mnt`	permanent mount points for extra storage
`/opt`	optional third-party software — Firefox binary lives here
`/run/media`	where GNOME auto-mounts drives temporarily when you click them

Your specific drive layout — Lenovo IdeaPad Slim 3

```
nvme0n1 — your physical 512GB NVMe SSD
nvme0n1p1 → /boot/efi — boot files, tiny FAT32 partition
nvme0n1p3 → / — your Ubuntu 26.04 installation (ext4, 248GB)
nvme0n1p5 → /mnt/Saftech — your D: drive (NTFS, 88GB)
nvme0n1p6 → /mnt/Mamoni — your E: drive (NTFS, 138GB)
```